THENOTHING V7.1 · CLAUDE CODE MODE

HYDRA Offensive Knowledge Operating System

Project Documentation — Phase O: Temporal Knowledge Intelligence

15 phases (A–O) • **153** effective capabilities • **439** adapters • **7** agents • **108** MCP tools

Deterministic | Offline-first | Rebuild-identical | Advisory-only | Canonical-wiki-centered | Federation-safe | 499 tests green

Comprehensive architecture & design reference | Generated 2026-06-06
Single source of truth: docs/HYDRA_SYSTEM_CONTEXT.md (architecture memory)

Table of Contents

1 Project Overview

HYDRA is an **Offensive Knowledge Operating System** — a layered, pure-Python platform that an LLM operator drives through the **Model Context Protocol (MCP)** to perform high-quality bug-bounty and offensive-security **research within explicit authorization**. It is the engine behind **THENOTHING v7.1 — Claude Code Mode**.

The platform is built bottom-up across **15 locked phases (A → O)**. Each phase adds a *derived, advisory* intelligence layer on top of a single **canonical knowledge store** (the wiki). Nothing below the wiki ever writes back to it except through explicit, propose-only paths. Every derived layer is deterministic, offline-first, and rebuild-identical.

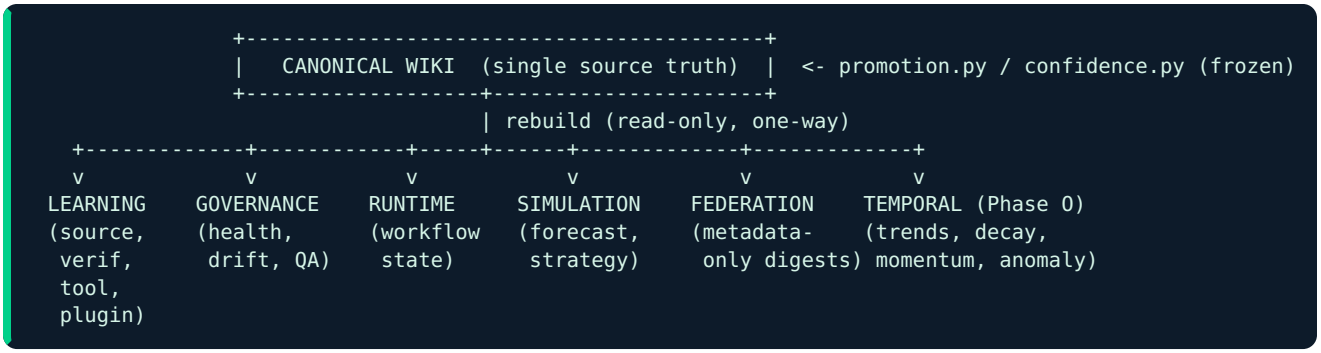
Property	Value
Current Phase	**O — Temporal Knowledge Intelligence**
Core capabilities	87
Effective capabilities	153 (core + 6 plugin packs)
Adapters	175 core / 439 effective
Agents	7
Plugins	6 reference packs
MCP tools	108
Tests	499 passing (6 integration deselected)
License	MIT

> **Design philosophy:** reason before executing, simulate before interacting, correlate evidence across domains, learn from every outcome — while never silently mutating the canonical wiki, the promotion rules, or the confidence engine.

2 Architecture Highlights

HYDRA's nine-phase reasoning loop sits on a stack of derived, deterministic subsystems:

> **Reasoning loop:** Observe -> Understand -> Reason -> Simulate -> Plan -> Execute -> Validate -> Learn -> Replan



- **Capability-first.** Everything is a *capability* (e.g. `subdomain_discovery`), realized by interchangeable *tools* via *adapters*.
- **Derived & rebuildable.** Every learning/intelligence store is a pure function of an append-only event log; delete it and it rebuilds identically.
- **Advisory-only.** Simulation, governance, federation and temporal layers *recommend*; they never execute, confirm, promote, or exploit.
- **Offline-first & deterministic.** Given a fixed injected clock, outputs are byte-identical (rebuild-identical).

3 Architecture Lineage (Phases A -> O)

Build order is **locked A -> O**. Every phase is derived/advisory unless noted; none mutate promotion.py, confidence.py, or canonical wiki behavior.

Phase	Theme	Purpose	MCP Δ
A	Offensive Knowledge OS Foundations	Capability-first recon + a machine-operable wiki as the single canonical store.	+10
B	Report Intelligence	Distill disclosed reports/writeups into reusable, scored attacker knowledge.	+3
C	Pattern & Chain Discovery (propose-only)	Cross-document synthesis into pattern/chain candidates.	+3
C.5	Scalability Hardening	Remove O(F^2) chain discovery; rebuild amplification fixes.	0
D / D.1	Source Performance Learning & Opportunity Ranking	Event-sourced learning to prioritize without touching canonical state.	+4
E	Adaptive Recon & Autonomous Source Selection	Use Phase-D learning to advise recon planning.	+2
F	Verification Learning & Validation Intelligence	Learn how findings get validated; advisory verification playbooks.	+4
G	Capability Expansion & Tool Orchestration	Capability-centric catalog v2 (87 caps / 9 categories) + learning tool selector.	+4
H	Multi-Agent Orchestration	Declarative agents over the capability layer; deterministic routing.	+4
I	Execution Runtime & Workflow Engine	Deterministic workflow STATE coordination (no execution). Adds mobile_agent -> 87/87 owned.	+4
J	Knowledge Governance, Drift & QA	Derived health/freshness/consistency evaluation.	+6
K	Adapter Framework & Sandboxed Tool Integrations	Capability x tool adapter definitions (175 core) + sandboxed runtime + tool-health learning.	+6
L	Autonomous Knowledge Simulation & Decision Intelligence	Predict workflow/plan/strategy outcomes from historical learning before execution.	+8
M	Capability Marketplace & Plugin Ecosystem	Declarative plugins extend capabilities/adapters/ agents. core(87)+plugins(66)=153 effective, 439 adapters.	+12
N	Federated Knowledge Exchange & Intelligence Mesh	Exchange anonymized, aggregated intelligence digests between instances - metadata only.	+10

Phase	Theme	Purpose	MCP Δ
O	Temporal Knowledge Intelligence	Understand how knowledge evolves over time: trends, momentum, decay, forecasts, anomalies.	+6

Per-phase detail

Phase A — Offensive Knowledge OS Foundations

MCP +10

Purpose. Capability-first recon + a machine-operable wiki as the single canonical store.

Components. CapabilityRegistry, WikiStore, recon-fusion (Two-Signal confidence), graph index, promotion.py, confidence.py, safety/test harness.

Invariants preserved. Wiki canonical; promotion/confidence introduced and frozen thereafter.

Phase B — Report Intelligence

MCP +3

Purpose. Distill disclosed reports/writeups into reusable, scored attacker knowledge.

Components. ReportIntelligencePipeline; report+intel pages only; deterministic 1-10 learning score.

Invariants preserved. No findings/patterns/chains created; LLM-free deterministic scoring.

Phase C — Pattern & Chain Discovery (propose-only)

MCP +3

Purpose. Cross-document synthesis into pattern/chain candidates.

Components. PatternDiscovery, ChainDiscovery, evidence weighting, confirm_candidate (only write path).

Invariants preserved. Discovery is dry-run; canonical pages only via explicit confirm.

Phase C.5 — Scalability Hardening

MCP 0

Purpose. Remove O(F^2) chain discovery; rebuild amplification fixes.

Components. Bounded chain discovery; canonical signatures from structured fields only.

Invariants preserved. Determinism preserved; complexity reduced to O(E).

Phase D / D.1 — Source Performance Learning & Opportunity Ranking

MCP +4

Purpose. Event-sourced learning to prioritize without touching canonical state.

Components. SourceLearningStore, OpportunityScorer; D.1 robustness (no dead-zones).

Invariants preserved. Learning derived/rebuildable; never touches promotion/confidence.

Phase E — Adaptive Recon & Autonomous Source Selection

MCP +2

Purpose. Use Phase-D learning to advise recon planning.

Components. AdaptiveSourceSelector, ReconPlanner.

Invariants preserved. Advisory; recommends, never executes/confirms/writes.

Phase F — Verification Learning & Validation Intelligence

MCP +4

Purpose. Learn how findings get validated; advisory verification playbooks.

Components. VerificationLearningStore, ValidationIntelligence, PlaybookGenerator, ToolCapabilityRegistry.

Invariants preserved. Never auto-confirms/auto-exploits; WAL, idempotent.

Phase G — Capability Expansion & Tool Orchestration

MCP +4

Purpose. Capability-centric catalog v2 (87 caps / 9 categories) + learning tool selector.

Components. CapabilityCatalog, CapabilityCoverage, ToolSelector.

Invariants preserved. Capability modeling only; no integrations; read-only.

Phase H — Multi-Agent Orchestration

MCP +4

Purpose. Declarative agents over the capability layer; deterministic routing.

Components. AgentRegistry (6 agents), AgentPlanner; Target->Agent->Capability->Tool.

Invariants preserved. Agents never execute/confirm/write; advisory.

Phase I — Execution Runtime & Workflow Engine

MCP +4

Purpose. Deterministic workflow STATE coordination (no execution). Adds mobile_agent -> 87/87 owned.

Components. RuntimeEngine, WorkflowStore (workflows.db).

Invariants preserved. Executes no tools; materializes nothing canonical.

Phase J — Knowledge Governance, Drift & QA

MCP +6

Purpose. Derived health/freshness/consistency evaluation.

Components. DriftDetector, KnowledgeQualityAnalyzer, GovernanceIntelligence (knowledge_governance.db).

Invariants preserved. Read-only; writes nothing canonical.

Phase K — Adapter Framework & Sandboxed Tool Integrations

MCP +6

Purpose. Capability x tool adapter definitions (175 core) + sandboxed runtime + tool-health learning.

Components. AdapterRegistry, ToolHealthStore, AdapterIntelligence, CapabilityExerciseAnalyzer.

Invariants preserved. No offensive execution; only SAFE profiles; unsupported profiles rejected at load.

Phase L — Autonomous Knowledge Simulation & Decision Intelligence

MCP +8

Purpose. Predict workflow/plan/strategy outcomes from historical learning before execution.

Components. SimulationContext (load-once O(E)), WorkflowSimulator, OutcomePredictor, PredictionAnalytics.

Invariants preserved. Advisory; governance gains decision_intelligence block; no execution/promotion.

Phase M — Capability Marketplace & Plugin Ecosystem

MCP +12

Purpose. Declarative plugins extend capabilities/adapters/agents. core(87)+plugins(66)=153 effective, 439 adapters.

Components. PluginRegistry, EffectiveCapabilityCatalog, CapabilityDependencyGraph, AgentOwnershipResolver.

Invariants preserved. Globally-unique capability ids; no plugin execution; promotion/confidence untouched.

Phase N — Federated Knowledge Exchange & Intelligence Mesh

MCP +10

Purpose. Exchange anonymized, aggregated intelligence digests between instances - metadata only.

Components. federation/: safety guard, KnowledgeExchangeStore, FederationRegistry, KnowledgeDigestGenerator, IntelligenceMesh, ConsensusEngine, FederationMarketplace.

Invariants preserved. Metadata-only (assert_safe on export+import); advisory; promotion/confidence untouched.

Phase O — Temporal Knowledge Intelligence

MCP +6

Purpose. Understand how knowledge evolves over time: trends, momentum, decay, forecasts, anomalies.

Components. temporal_intel/: TemporalStore, TemporalContext (load-once, memoized), TrendAnalyzer, MomentumAnalyzer, TemporalForecastEngine, DecayAnalyzer, TemporalAnomalyDetector, TemporalAdvisor, TemporalIntelligence.

Invariants preserved. Derived/advisory/deterministic; no wiki mutation; promotion/confidence untouched.

4 The Seven Agents

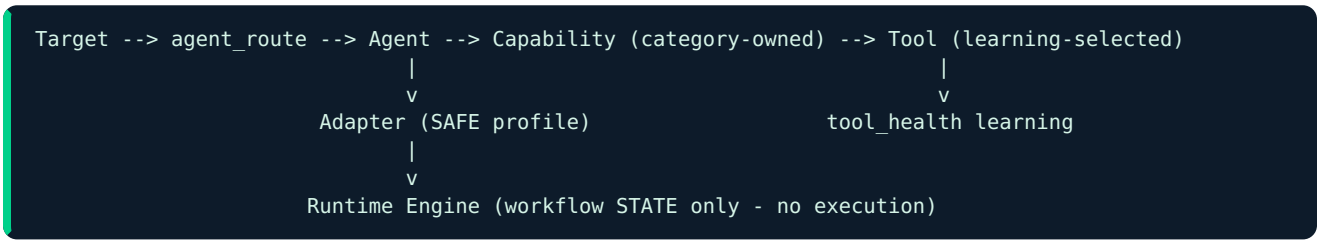
Introduced in **Phase H** (declarative agents) and made executable-as-*state* in **Phase I** (runtime), the agents form a deterministic routing layer. A target is routed to the right **agent**, which owns a set of capability **categories**, each realized by a learning-selected **tool/adapter**. Agents **never execute tools, confirm findings, or write the wiki** — they plan and route; the runtime tracks state only.

Agent	Priority	Owns categories	Responsibility
recon_agent	10	reconnaissance, infrastructure	Discover and map the external attack surface - subdomains, DNS, ASN/CIDR, ports, services, tech, TLS.
attack_surface_agent	8	web, api	Enumerate the web/API surface and probe for vulnerability candidates (advisory).
cloud_agent	7	cloud, secrets, source_code	Discover cloud assets, leaked secrets, and source-code / dependency exposures.
verification_agent	6	verification	Validate suspected findings using verification playbooks - never auto-confirms.
mobile_agent	6	mobile	Static-analyze mobile apps (APKs): secret/ endpoint extraction, deeplink and cert-pinning checks. Added in Phase I - closes capability ownership to 87/87.
correlation_agent	5	(cross-cutting)	Correlate findings and report-intel into patterns and chains - propose-only; promotion rules unchanged.
reporting_agent	3	(cross-cutting)	Synthesize confirmed knowledge into structured bug-bounty reports.

Expected outputs per agent

Agent	Expected output asset types
recon_agent	subdomain, dns_record, ip, asn, open_port, service, technology, host
attack_surface_agent	endpoint, parameter, url, xss, sqli, ssrf, graphql, vulnerability
cloud_agent	cloud_bucket, cloud_misconfig, secret, credential, repository, dependency_vuln
verification_agent	idor, ssrf, auth_bypass, open_redirect, csrf, xss, sqli, takeover
mobile_agent	mobile_finding, secret, credential, endpoint, internal_host, deeplink, cert_pinning
correlation_agent	pattern, chain
reporting_agent	report

How agents interact with the stack



Routing is deterministic: `agent_route` maps Target→Agent→Capability→Tool. Quality is observable via `agent_coverage` (orphans / overlaps / bottlenecks) and `agent_effectiveness` (Phase-L multi-agent simulation). The effective catalog is **153/153 owned, with zero gaps or conflicts**.

5 MCP Tools (108)

All tooling is exposed to the LLM operator through the **hydra-security** MCP server (`python mcp_server.py`, stdio or SSE). A committed contract baseline and a CLAUDE.md doc-sync test guard against registry drift.

Layer (phase)	Representative tools
Recon & Surface	`subfinder_scan, httpx_probe, katana_crawl, gau_urls, dnsx_resolve, amass_enum, nmap_scan, full_recon, check_tools`
Vulnerability & Fuzzing	`nuclei_scan, nuclei_scan_list, sqlmap_scan, dalfox_scan, gxss_check, ffuf_fuzz, dirsearch_scan`
Fingerprinting	`whatweb_detect, wafw00f_detect`
Knowledge OS (A-C)	`recon_fuse, kb_recall, kb_lint, kb_promote, kb_rebuild_index, asset_lookup, graph_neighbors, graph_path, ingest_report, report_lookup, list_reports, discover_patterns, discover_chains, confirm_candidate`
Learning & Ranking (D-F)	`record_outcome, source_scores, rank_opportunities, prioritization_report, select_sources, recon_plan, record_verification, verification_stats, verification_playbook, tool_capabilities`
Capabilities & Agents (G-H)	`capability_catalog, capability_coverage, rank_tools, select_tool, agent_catalog, agent_plan, agent_route, agent_coverage`
Runtime & Governance (I-J)	`workflow_create, workflow_status, workflow_history, runtime_summary, governance_summary, drift_report, knowledge_health, stale_entities, duplicate_patterns, contradiction_report`
Adapters & Simulation (K-L)	`adapter_catalog, adapter_coverage, adapter_health, adapter_summary, adapter_select, runtime_analytics, simulate_workflow, simulate_strategy, predict_outcome, capability_impact, prediction_accuracy, agent_effectiveness, workflow_optimization, decision_health`
Marketplace (M)	`plugin_catalog, plugin_summary, plugin_health, plugin_dependencies, plugin_capabilities, plugin_coverage, capability_graph, dependency_paths, critical_capabilities, agent_ownership, ownership_conflicts, ecosystem_summary`
Federation (N)	`federation_peers, federation_summary, export_digest, import_digest, capability_trends, verification_trends, source_trends, federation_consensus, ecosystem_opportunities, federation_health`
Temporal (O)	`temporal_summary, temporal_trends, temporal_forecast, temporal_decay, temporal_anomalies, temporal_health`

Beyond the Knowledge-OS phase tools (A-O), the registry also includes the legacy operational palette (recon/scan/fuzz) and base tools (save_finding, get_findings, generate_report). Live total: **108**.

6 Capabilities, Adapters & Plugins

Capabilities

A **capability** is an abstract objective (e.g. `subdomain_discovery`, `port_scanning`, `xss_probing`) independent of any specific tool. The core catalog holds **87** capabilities across 9 categories; six declarative plugin packs raise the **effective** catalog to **153**, with globally-unique ids (duplicates rejected).

Adapters

An **adapter** binds one capability to one tool (`capability::tool`) with a sandboxed, deterministic definition: execution profile, timeouts, I/O schemas. Only **SAFE profiles** (offline / passive / validation / simulation) load; exploitation / persistence / destructive / weaponized profiles are rejected. The effective adapter count is **439** (175 core).

Plugins (6 reference packs)

Pack	Adds
cloud	cloud asset / misconfig capabilities
mobile	mobile (APK) analysis capabilities
container	container / K8s exposure capabilities
iot	IoT surface capabilities
supply_chain	dependency / supply-chain capabilities
osint	OSINT enrichment capabilities

core_capabilities + plugin_capabilities = effective_capability_catalog. A capability dependency graph (requires / enhances / related_to) is acyclic-validated; agent ownership is automatic and complete (153/153).

7 Stores & Databases

Every store under `data/` is **derived, disposable, gitignored**, and rebuildable from its append-only event log (WAL mode, idempotent writes). None is a second canonical source.

Layer	Database	Purpose
Learning	`source_learning.db`	Source performance learning (trust / effectiveness / novelty)
Learning	`source_metrics.db`	Per-source run metrics
Learning	`verification_learning.db`	Verification method / evidence success learning
Learning	`tool_health.db`	Adapter tool-health (reliability / runtime / outcomes)
Learning	`plugin_health.db`	Plugin/capability usage learning
Intelligence	`decision_learning.db`	Simulation / forecasting / strategy (Phase L)
Intelligence	`temporal.db`	Temporal evolution: trends / decay / forecasts (Phase O)
Runtime	`workflows.db`	Deterministic workflow state (Phase I)
Governance	`knowledge_governance.db`	Health / drift snapshots (Phase J)
Federation	`federation.db`	Append-only metadata-only exchange ledger (Phase N)
Canonical index	`knowledge_index.db`	Derived graph index (rebuildable from the wiki)

8 Architecture Invariants

These invariants are enforced in code and **continuously verified by the test suite** (e.g. promotion/confidence immutability checks, no-wiki-mutation tests, metadata-only federation guards, rebuild-identical determinism tests).

Area	Invariant
Canonical knowledge	Wiki is the single canonical source; no second canonical source; no dual-write.
Protected core	promotion.py and confidence.py are immutable (last touched in Phase A).
Discovery	Propose-only; no autonomous confirmation; no autonomous promotion.
Learning	Derived, disposable, event-sourced, rebuild-identical.
System	Offline-first; deterministic; advisory-only decision systems; MCP backward-compatible.
Execution	No autonomous exploitation; no offensive execution; SAFE adapter profiles only.
Federation	Metadata only - never findings, evidence, targets, sources, or secrets.
Marketplace	Declarative plugins only - no plugin execution.
Confidence	No hidden confidence modification or rewriting.

Invariant regression score at Phase O: 100% preserved (0 regressions).

9 Data Flow & Architecture Graph

Data flow map

```
recon-fusion --+                                  +-- ingest_report / confirm_candidate
               v                                  v     (explicit, propose-only writers)
       +---------------------- CANONICAL WIKI ---------------------+
       |            promotion.py . confidence.py  (frozen)         |
       +-------------------------------+-------------------------+
                                       | rebuild (read-only)
                                       v
                              knowledge_index.db (graph)
                                       | read-only
  +------------+------------+--------+--------+------------+------------+
  v            v            v                 v            v            v
LEARNING    GOVERNANCE     RUNTIME         SIMULATION     FEDERATION     TEMPORAL
 stores      snapshots      state           forecasts      digests        trends/decay

RULE: every arrow below the wiki is READ-ONLY derived; nothing writes back to
      canonical except the explicit propose-only writers at the top.
```

Architecture relationship graph

```
Capabilities (87 core / 153 effective)
  |-- owned by ----> Agents (7)            [Phase H/I, deterministic routing]
  |-- realized by -> Adapters (175/439)    [Phase K, SAFE profiles only]
  |                     '-- health ------> Adapter Intelligence (advisory)
  |-- extended by -> Plugins (6 packs)     [Phase M, declarative + dependency graph]
  |-- planned by --> Runtime Engine        [Phase I, workflow STATE only]
  |-- predicted by-> Simulation            [Phase L, advisory forecasts]
  |-- evaluated by-> Governance            [Phase J, read-only health/drift]
  |-- shared by ---> Federation            [Phase N, metadata-only digests]
  '-- tracked by --> Temporal Intelligence [Phase O, trends/decay/forecast/anomaly]
```

10 Performance History & Benchmarks

Every derived layer targets **O(E)** (linear in event count); no O(N²). Capability/MCP growth across phases:

Phase	Caps	Adapters	Agents	MCP	Scaling characteristic
A	87	-	-	10	recon fusion; index rebuild
C.5	87	-	-	16	removed O(F^2) -> O(E)
D/D.1	87	-	-	20	O(E) event-sourced learning
F	87	-	-	26	O(E), WAL idempotent
H	87	-	6	34	deterministic routing
I	87	-	7	38	O(steps) workflow state
K	87	175	7	50	O(E) tool-health
L	87	175	7	58	O(E) load-once SimulationContext
M	153	439	7	70	O(C) incremental plugin loading
N	153	439	7	102	O(E); ~2 us/event federation reads
O	153	439	7	**108**	O(E); 9.35 s @ 1M rows (2M events)

Phase-O temporal benchmark (load-once context, memoized bucketing)

Events (rows)	ctx build	trends	summary	TOTAL	µs/event
10,000	0.04 s	0.30 s	0.27 s	0.61 s	61.5
100,000	0.91 s	0.38 s	0.21 s	1.51 s	15.1
1,000,000	5.96 s	1.78 s	1.61 s	**9.35 s**	**9.35**

Per-event cost **decreases then flattens** (61 -> 15 -> 9 µs) as fixed overhead amortizes — confirming linear O(E) scaling with no performance cliff. 1M rows = 2M derived events (worst case: one high-volume table feeding two domains).

Federation benchmark (Phase N)

Read cost flat at **~1.6–2.0 µs/event across 2k–40k events** — O(E), rebuild-identical digests.

11 Open Risks

Risk history is never deleted — items move Open → Mitigated → Closed.

Class	Item	Status
Architectural debt	Branch/tag drift; no root INDEX.md; source_learning vs source_metrics overlap.	Open
Scaling	Mesh/consensus re-read patterns; no event-log compaction beyond ~1e6.	Open
Performance	Per-store SQLite connections, no shared pool (fixed fan-out cost).	Open
Coverage	Exercise coverage is cold-start low until learning stores fill.	Mitigated
Future	Apply load-once context pattern to federation; version plugin deps across peers.	Open

12 Roadmap (Phase O -> Z)

All future phases inherit every A–O invariant: derived, deterministic, offline-first, advisory-only, rebuildable, canonical-wiki-centered.

Phase	Goal	Status	MCP impact
O	Temporal Knowledge Intelligence	DELIVERED (current)	+6 (108)
P	Unified Intelligence & Cross-Store Correlation Layer	next	+~4
Q	Federated Trust Graph & Reputation Hardening	planned	+~4
R	Reporting & Deliverable Synthesis Intelligence	planned	+~3
S	Knowledge Compaction & Snapshotting	planned	+~2
T	Multi-Tenant Scope & Isolation	planned	+~3
U	Observability & Audit Intelligence	planned	+~3
V	Capability Confidence Calibration (advisory)	planned	+~2
W	Federated Simulation Exchange	planned	+~3
X	Adaptive Roadmap & Self-Planning Intelligence	planned	+~2
Y	Cross-Ecosystem Interop & Schema Federation	planned	+~2
Z	Architecture Self-Audit & Invariant Enforcement Engine	planned	+~3

13 Validation & Quality

Dimension	Result
Total tests	499 passing (6 integration/e2e deselected)
Temporal (Phase O)	23 (18 unit + 5 MCP)
Federation (Phase N)	26 (19 unit + 7 MCP)
Determinism	rebuild-identical under injected clock (digest + summary byte-identical)
MCP contract	108 live = 108 baseline = 108 documented; 0 orphans/dupes/missing
Invariants	promotion/confidence immutable; no-wiki-mutation; metadata-only federation
Lint	ruff clean across all phase surfaces

The project follows an **Architecture Steward protocol**: every phase passes a design review, an invariant/safety gate, benchmarks, and an architecture-memory update before it is considered complete.

14 Appendix — Glossary & Maintenance

Term	Meaning
Capability	An abstract objective realized by interchangeable tools.
Adapter	A capability x tool binding with a SAFE execution profile.
Agent	A declarative owner of capability categories that plans & routes (never executes).
Canonical wiki	The single source of truth; only propose-only writers may modify it.
Derived store	An event-sourced, disposable, rebuildable learning/intelligence database.
Two-Signal rule	Confidence elevates only with two independent corroborating signals.
MCP	Model Context Protocol - how the LLM operator invokes HYDRA's 108 tools.
Rebuild-identical	Deleting and replaying a derived store yields byte-identical state.

Maintenance protocol

After any phase completes, the architecture memory (`docs/HYDRA_SYSTEM_CONTEXT.md`), `CLAUDE.md`, and the MCP contract baseline are updated automatically with the new counts, benchmarks, tests, risks, and roadmap. The memory file is a first-class architecture artifact.